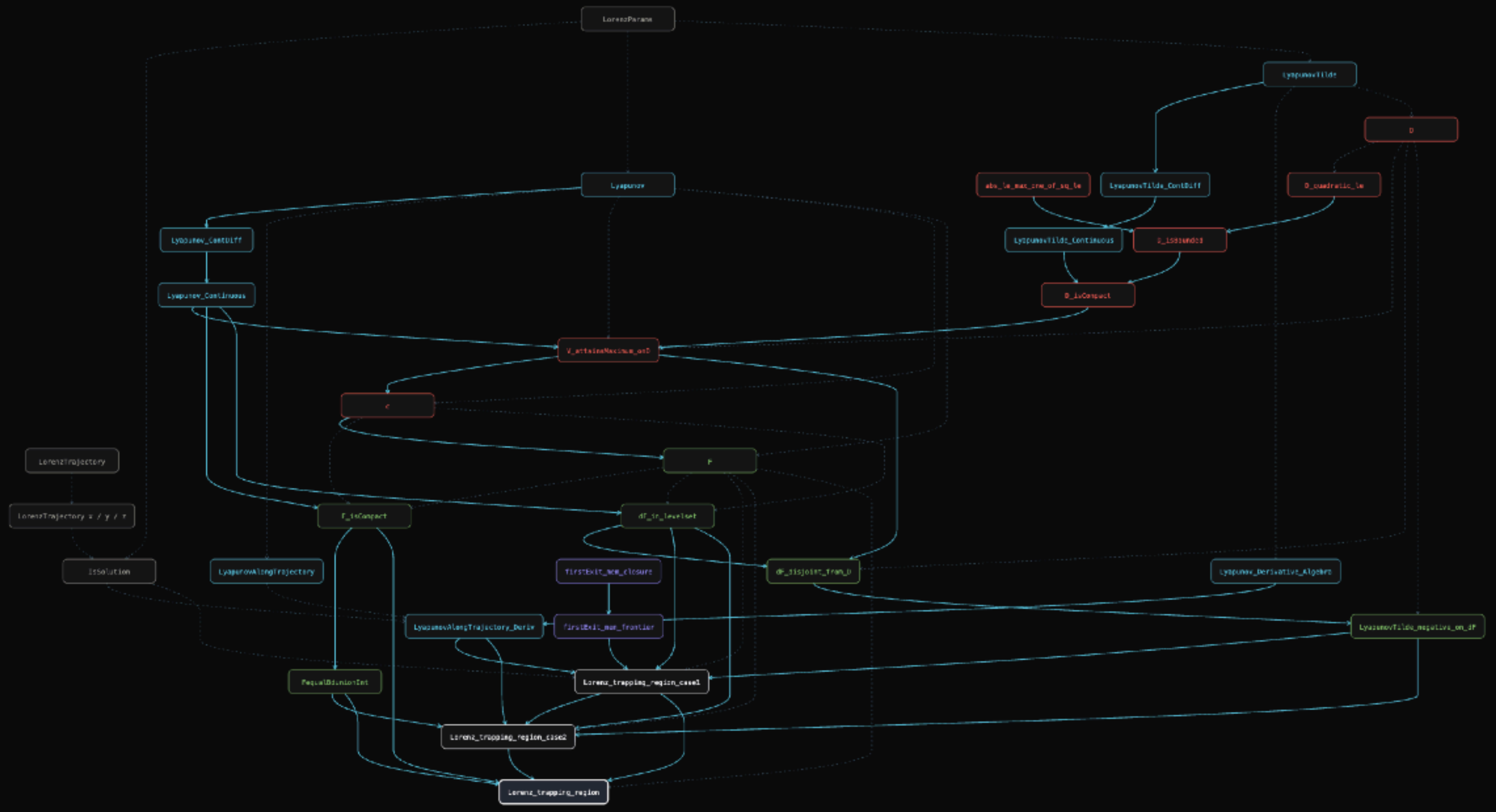
- Phase 1 · The dynamical system
- Phase 2 · Defining V and Ṽ
- Phase 3 · The set D and sup c
- Phase 4 · The set E and ∂E
- Toolkit · replaces Case 1 subcases 1–2
- Phase 5 · Trapping region



STEP 31 OF 31 · THEOREM · **PHASE 5 · THEOREM**

Lorenz_trapping_region

"If $\phi(t_0) \in E$, then $\phi(t) \in \mathrm{int}(E)$ for all $t > t_0$." Four moves: **use F p** to supply the witness, **F_isCompact.isBounded** to discharge the boundedness conjunct, **FequalBdunionInt** to split F into interior ∪ frontier, and then each piece to its case. Note that the conclusion is **stronger than the hypothesis**: land anywhere in F — even exactly on the boundary — and at every later instant you are strictly inside. The trapping region does not merely hold you; it pulls you off its own edge.

```
theorem Lorenz_trapping_region (p) (φ)
    (hφ : IsSolution p φ) :
  ∃ F : Set (ℝ × ℝ × ℝ),
    Bornology.IsBounded F ∧
    ∀ t0 : ℝ, φ.total t0 ∈ F →
      (∀ t > t0, φ.total t ∈ interior F) := by
  use F p
  rw [and_iff_left ?_]
  · refine IsCompact.isBounded ?_
    apply F_isCompact
  · intro t0 h1
```

Next Steps

1. **Contribute the boundary lemmas to Mathlib.**

   Goal. Generalize and upstream the auxiliary topological results developed for the trapping region proof.

   - `firstExit_mem_frontier` *The first exit time lies on the frontier of the set.*
   - `firstExit_mem_closure` *A right-continuous flow maps the first exit time into the closure of the complement.*

   These lemmas are completely independent of the Lorenz system and are potentially useful in broader formalizations.

2. **Extend the formalization of the Lorenz system.**

   Prove additional fundamental results, including
   - Global Existence Theorem
   - Existence of an Attracting Set
   - Further qualitative properties of the Lorenz flow.

3. **Generalize the trapping region lemma.**

   Develop a version applicable to more general dynamical systems, including
   - Higher-dimensional systems
   - Non-smooth flows
   - Infinite-dimensional systems

$\Rightarrow$ Towards a reusable formal library for trapping regions and dynamical systems in Lean.